

IBM University Engagement Project Submission

“Exploring the Power of Agentic AI with IBM Granite and LangFlow”

Project Title – [AI-Powered LaTeX Diagram Generator for Academic Research]

Domain of Project – [like Agriculture / Education / Healthcare / ...]

Student Name	Recent Passport Photo		Email ID	Mobile number [WhatsApp]
Mohammed Omer Farooq			mohdomerfarooq17@gmail.com	+91 7842376426

Problem Statement: AI-Powered LaTeX Diagram Generator for Academic Research

The Challenge

Academic researchers and students frequently need professional diagrams for publications, theses, and presentations. Creating diagrams in LaTeX using TikZ requires deep technical expertise, significant time investment, and manual coding of complex syntax. Existing tools either lack **publication-quality output** or require expensive proprietary software. Without an intelligent, accessible solution, researchers waste valuable time on diagram formatting instead of focusing on their **core research work**.

The Objective

Build an **AI-Powered LaTeX Diagram Generator** that automatically converts plain-English descriptions into professional TikZ code for LaTeX documents. The solution should:

Natural Language Input – Accept plain-English diagram descriptions without requiring any LaTeX or TikZ knowledge from the user.

TikZ Code Generation – Automatically produce valid, compilable TikZ code using IBM Granite code models via watsonx.ai.

Real-Time Preview – Render the generated diagram instantly inside IBM Watson Studio without any external tools or compilers.

Plain-English Refinement – Allow iterative diagram improvement using natural language commands like “add blue color to the Lexer node.”

Multi-Diagram Type Support –

Flowcharts (compilation pipelines, process flows)

Block Diagrams (system architectures, layered models)

State Machines (FSMs, automata, transition diagrams)

Trees & Hierarchies (binary trees, org charts, decision trees)

Timelines, Venn Diagrams, Sequence Diagrams

Publication-Ready Output – Generate LaTeX-compatible TikZ code suitable for journals, IEEE papers, conference submissions, and academic theses.

Technology Used

Tech Stack (Mandatory)

Use **any one** of the following approaches:

IBM Watson Studio + IBM watsonx.ai + Granite Models

IBM Cloud Agentic Lab with IBM Granite Models

LangFlow + IBM watsonx.ai + Granite Models

I used **IBM Watson Studio + IBM watsonx.ai + Granite Models**

Specifically:

IBM Watson Studio – Jupyter Notebook development and execution environment

IBM watsonx.ai Runtime – Model deployment, inference, and AI governance platform

IBM Granite 8B Code – ibm/granite-8b-code-instruct for TikZ code generation

ibm-watsonx-ai SDK – Python SDK for watsonx.ai model API calls

matplotlib + networkx – Local diagram rendering inside the Watson Studio notebook

Python 3.12 – Core programming language

Proposed Solution -

Proposed Solution: AI-Powered LaTeX Diagram Generator

The proposed solution is an **AI-powered LaTeX diagram generation agent** built using **IBM Watson Studio, IBM watsonx.ai, and IBM Granite 8B Code Instruct** to provide researchers with instant, publication-ready TikZ diagrams from plain-English descriptions. At its core, the solution uses a **prompt-engineered generation pipeline** where the user's natural language description is passed to the Granite 8B Code model, which has been instruction-tuned for code generation tasks. The model outputs valid, compilable TikZ code following LaTeX best practices including proper node naming, library usage, and diagram structure. The generated code is then rendered as a visual diagram directly inside the Watson Studio notebook using matplotlib and networkx, eliminating the need for any external LaTeX compiler or third-party tool.

The system consists of four key components:

Prompt Engineering Layer structures the user's plain-English input into a precise instruction prompt that guides the Granite model to produce syntactically correct TikZ code with no prose or markdown fences.

TikZ Code Generation Engine uses IBM Granite 8B Code Instruct via the watsonx.ai Runtime API to convert natural language into raw TikZ code supporting flowcharts, state machines, trees, block diagrams, timelines, Venn diagrams, and sequence diagrams.

Multi-Turn Refinement System maintains the generated TikZ code in state and passes it back to the Granite model along with the user's plain-English refinement command, enabling iterative improvements like color changes, layout adjustments, and style modifications.

Local Rendering Engine parses the generated TikZ code and renders a visual diagram preview directly inside the Watson Studio Jupyter Notebook using matplotlib, with full support for colored nodes, directional arrows, and labeled edges — requiring zero external dependencies.

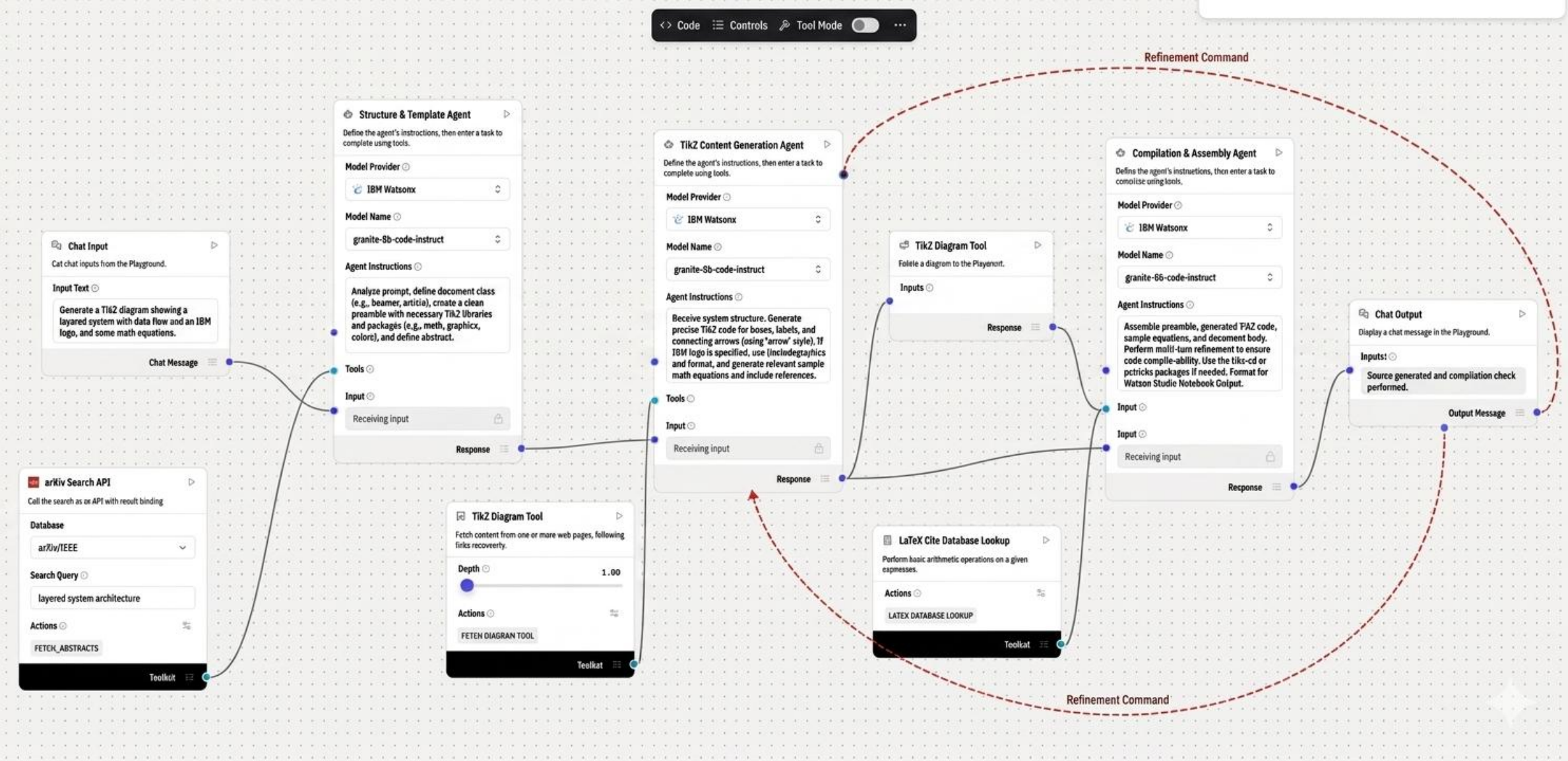
The solution includes a **cell-based interactive interface** that allows users to describe a diagram, view the generated TikZ code, preview the rendered output, and refine it — all within a single IBM Watson Studio notebook. IBM Watson Studio provides the managed collaborative environment, while watsonx.ai ensures scalable model inference and AI governance. Granite models power accurate code generation and context-aware refinement.

Overall, this solution delivers an **end-to-end intelligent diagram assistant** that eliminates LaTeX complexity for researchers and enables publication-quality diagrams in seconds.

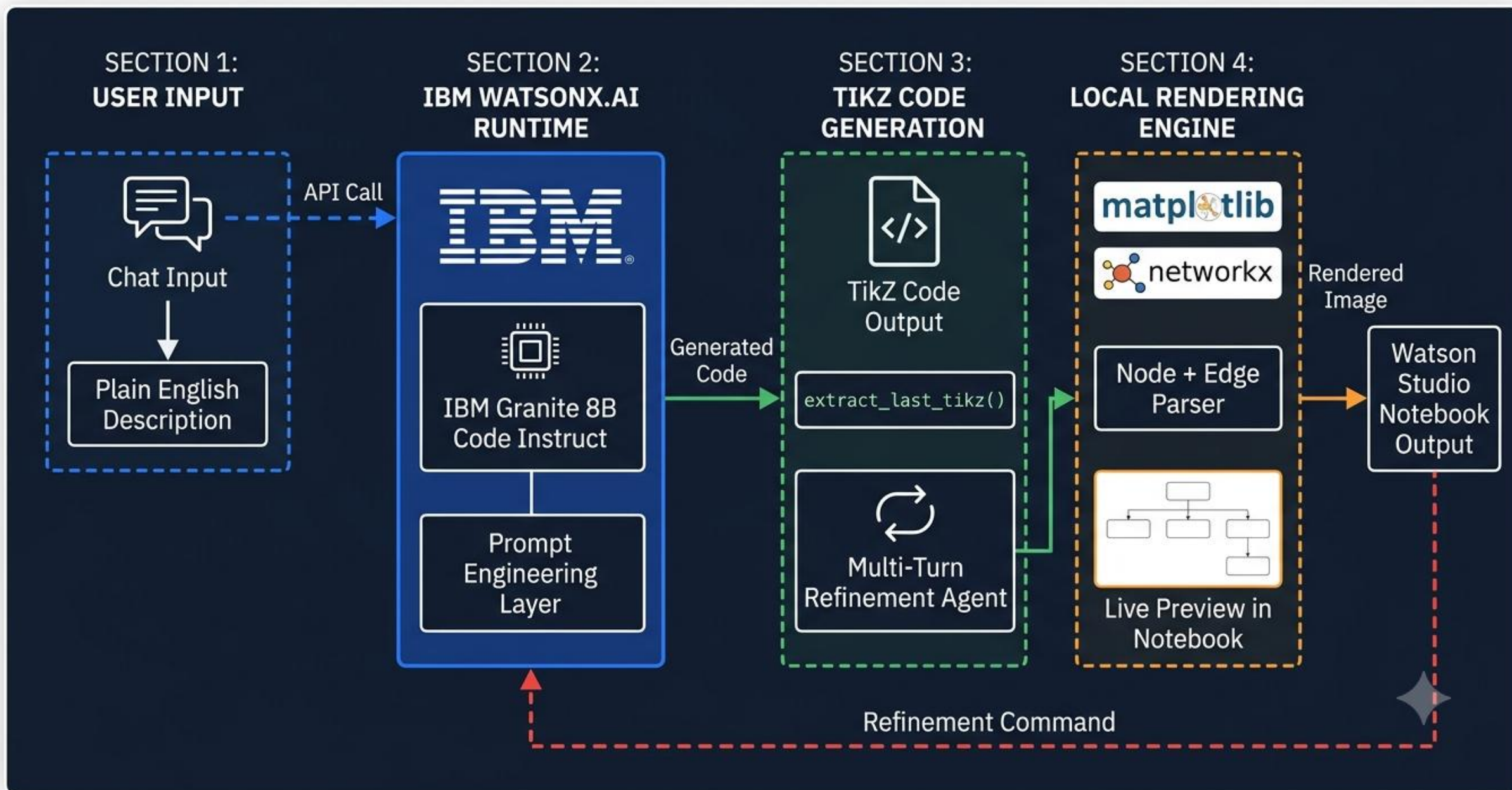
Langflow component Used -

- 1) **Chat Input** – Interface where users enter their queries, preferences, or meal details (text/voice/image).
- 2) **IBM watsonx AI Agent** - Processes user input using Granite models to generate personalized nutrition insights and recommendations.
- 3) **Name of IBM Granite model** - Granite-4.0-8B-Instruct → Fast, efficient for real-time diet recommendations

Langflow workflow -



Architecture blueprint



Role of Agentic AI in the solution

Role of Agentic AI in the LaTeX Diagram Generator

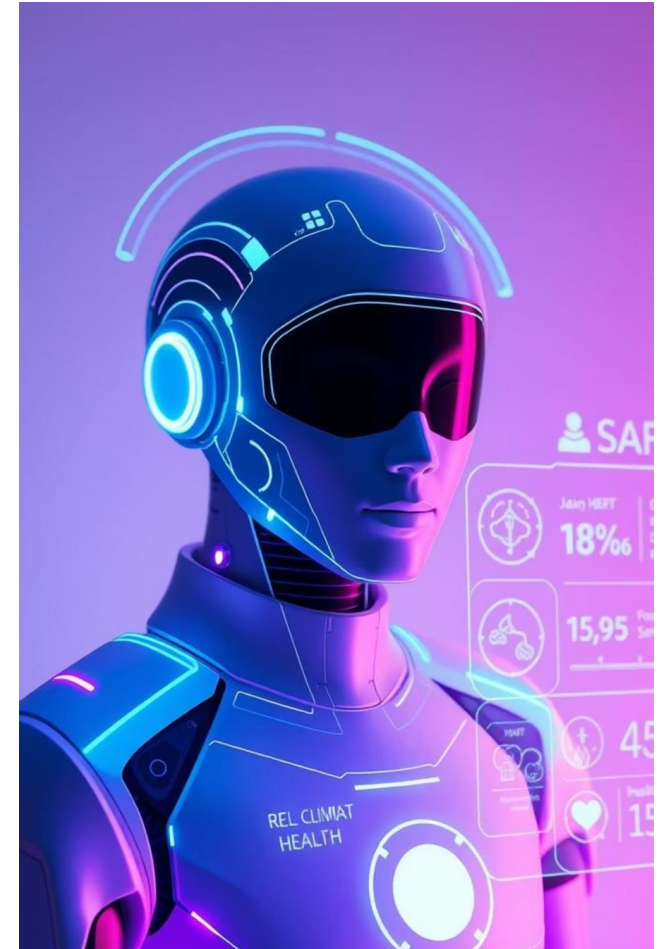
Agentic AI enables the LaTeX Diagram Generator to function as an intelligent, autonomous system rather than a simple code lookup tool. Using **IBM watsonx.ai** and **IBM Granite models**, it reasons over natural language input and autonomously decides how to structure, style, and generate a complete TikZ diagram.

The agent performs goal-directed behaviour — given a high-level description like “A flowchart showing the compilation pipeline,” it independently determines node layout, arrow direction, label naming, and TikZ library selection without any manual instruction. This **autonomous decision-making** is what separates it from a template-based tool.

Agentic AI also brings **context awareness**, retaining the current diagram state across multiple turns so that refinement commands like “add blue color to the Lexer node” are applied correctly to the existing diagram rather than generating a new one from scratch. It supports **real-time adaptation**, continuously updating the TikZ output based on user feedback in each notebook cell.

Additionally, the Granite model communicates the full diagram context back to itself in multi-turn prompts, enabling better structural decisions and consistent output across refinement iterations.

Overall, Agentic AI makes the system proactive, context-sensitive, and goal-driven, acting like a **smart diagram assistant** that helps researchers produce publication-quality LaTeX diagrams without writing a single line of TikZ code.



Agent Configuration & Knowledge Base :

IBM watsonx Orchestrate

EnglishContact SalesBuy Now30 trial days left

Home / Manage agents / LATEX CODE GENERATOR

LATEX CODE GENERATOR

Model : GPT-OSS 120B — Ope...

Profile

Knowledge

Toolset

Behavior

Channels Preview

Knowledge

Knowledge sources

Latex generation training...

Used by 1 agent

Edited 14 minutes ago

Toolset

Tools

Build with AITalk to agent

Running in debug mode

6:36 PM

Welcome! Describe any diagram and I'll generate professional TikZ/LaTeX code for you.

Accuracy of generated answers may vary. Please double-check responses.

Generate a decision tree for loan approval.

Create a neural network diagram with 3 input node...

Draw a client-server architecture with a web...

1/2

Create a flowchart for a login system.

Quick Start Prompts and Configuration :

IBM watsonx Orchestrate

EnglishContact SalesBuy Now30 trial days left

Home / Manage agents / LATEX CODE GENERATOR

LATEX CODE GENERATOR

Model : GPT-OSS 120B — Ope...

Build with AITalk to agent

Running in debug mode

Profile

Knowledge

Toolset

Behavior

Channels Preview

Quick start prompts

Pre-set messages sent to the agent to start the conversation.

Generate a decision tree for loan approval.

Create a neural network diagram with 3 input nodes, 2 hidden layers, and 1 output node.

Draw a client-server architecture with a web server, application server, and database.

See more +

Agent style

Define how your agent interprets prompts and decide what action to take.

Default Recommended

Relies on the models intrinsic ability to understand, plan and call tools and knowledge.

ReAct

Enables the model to think, act, observe and refine its approach until a task is completed.

Good for: High stakes apps

Voice modality Preview

6:47 PM

Welcome! Describe any diagram and I'll generate professional TikZ/LaTeX code for you.

Accuracy of generated answers may vary. Please double-check responses.

Generate a decision tree for loan approval.

Create a neural network diagram with 3 input node...

Draw a client-server architecture with a web...

< 1/2 >

Type something...

35°C Mostly sunny

Search

ENG IN

06:48 PM 13-07-2026

TESTING :

IBM watsonx Orchestrate

English

Contact Sales

Buy Now

30 trial days left

MO

Home / Manage agents / LATEX CODE GENERATOR

Test agent

live

Deploy

LATEX CODE GENERATOR

Model : GPT-OSS 120B — Ope...

Profile

Knowledge

Toolset

Behavior

Channels Preview

Knowledge

Provide the specific knowledge your agent needs to accomplish its goal.

Knowledge sources

Add an existing knowledge source, upload files, or connect to content repositories to add knowledge to your agent.

Replace source

+

Latex generation training...

by IBMid-6A7000FRRR

Latex generation training content

Used by 1 agent

Edited 14 minutes ago

Toolset

Add tools and other agents to help your agent take action.

Tools

Add tool

+

Build with AI

Talk to agent

Running in debug mode

You 6:37 PM

generate the Latex code and optimize it now

LATEX CODE GENERATOR 6:37 PM

Below is a compact, optimized LaTeX/TikZ source that draws a clean login-system flowchart. I have reduced redundancy, used a single `\tikzstyle` block, and set the node distances globally so the diagram scales nicely if you change the spacing later.

```
latex
% -----
% Login-system flowchart (stand-alone TikZ picture)
% -----
\documentclass[tikz,border=10pt]{standalone}
\usepackage{tikz}
\usetikzlibrary{shapes.geometric, arrows.meta, positioning}

\begin{document}
\begin{tikzpicture}[
  % ----- Styles -----
  startstop/.style = {rectangle, rounded corners, draw,
    fill=green!20, text width=5em,
    align=center, minimum height=1.2cm},
  process/.style    = {rectangle, draw,
    fill=blue!15, text width=6.5em,
    align=center, minimum height=1.2cm},
  io/.style          = {trapezium, top width=4em, bottom width=6em,
    draw, fill=yellow!10, align=center, minimum height=1.2cm}

  \node[startstop] (Start) {Start}
  \node[process] (Login) {Login}
  \node[process] (Auth) {Authentication}
  \node[process] (DB) {Database}
  \node[process] (Logout) {Logout}
  \node[io] (Exit) {Exit}

  Start --> Login
  Login --> Auth
  Auth --> DB
  DB --> Logout
  Logout --> Exit
  Exit --> Start

```

Save as test

Type something...

SCRIPT VERSION – USING IBM GRANITE MODELS :

IBM Cloud Pak for Data

Search in your workspaces

Upgrade ⓘ ⓘ Mohammed Omer Farooq's... Dallas MF

Projects / PS 26 LaTeX Diagram Generator / TikZ Generator

File Edit View Run Kernel Help

Not Trusted Memory:165 / 4096 MB Python 3.12

Renderer ready!

```
[11]: current_code = generate_tikz(
      description = "A flowchart: Source Code → Lexer → Parser → AST → Code Generator → Machine Code. Top-to-bottom layout.",
      diagram_type = "flowchart"
    )
print("--- TikZ Code ---")
print(current_code)
print("-----")
compile_and_show(current_code)

--- TikZ Code ---
\begin{tikzpicture}
\node (SourceCode) [startstop] {Source Code};
\node (Lexer) [process, below of=SourceCode] {Lexer};
\node (Parser) [process, below of=Lexer] {Parser};
\node (AST) [process, below of=Parser] {AST};
\node (CodeGenerator) [process, below of=AST] {Code Generator};
\node (MachineCode) [startstop, below of=CodeGenerator] {Machine Code};
\draw [arrow] (SourceCode) --> (Lexer);
\draw [arrow] (Lexer) --> (Parser);
\draw [arrow] (Parser) --> (AST);
\draw [arrow] (AST) --> (CodeGenerator);
\draw [arrow] (CodeGenerator) --> (MachineCode);
\end{tikzpicture}
-----
Nodes found : ['SourceCode', 'Lexer', 'Parser', 'AST', 'CodeGenerator', 'MachineCode']
Edges found : [('SourceCode', 'Lexer'), ('Lexer', 'Parser'), ('Parser', 'AST'), ('AST', 'CodeGenerator'), ('CodeGenerator', 'MachineCode')]
✓ Diagram rendered!
```

AI-Generated TikZ Diagram

```
graph TD;
  A[Source Code] --> B[Lexer];
  B --> C[Parser];
  C --> D[AST];
```

The diagram is a vertical flowchart with four rectangular nodes connected by downward-pointing arrows. The nodes are labeled 'Source Code', 'Lexer', 'Parser', and 'AST' from top to bottom. The 'AST' node has a line extending downwards from its bottom center, indicating the flow continues to the next step in the process.

Technology Used

- **Langflow platform** -For designing and orchestrating multi-agent workflows visually
- **IBM Watson Studio** - For developing, running, and demonstrating the full solution inside a managed Jupyter Notebook environment on IBM Cloud Pak for Data
- **IBM Granite 8B Code Instruct Model** - `ibm/granite-8b-code-instruct` – For natural language understanding, TikZ code generation, and context-aware diagram refinement
- **IBM Cloud** – Provides the infrastructure to build, deploy, and run the diagram generation agent reliably. IBM Cloud enables a **secure, managed, and scalable environment** for hosting Watson Studio and watsonx.ai Runtime
- **ibm-watsonx-ai SDK** – Python SDK used to **authenticate, connect, and call** the Granite model API from within the Watson Studio notebook, handling all model inference requests
- **IBM Watsonx.ai Runtime** - For model deployment, Granite model inference, and AI governance within the IBM Cloud ecosystem
- **matplotlib + networkx** (Local Rendering) – For **parsing TikZ code and rendering visual diagram previews** directly inside the Watson Studio notebook without any external LaTeX installation

Novelty and Uniqueness

The proposed LaTeX Diagram Generator stands out by combining **Agentic AI, code generation, and in-notebook rendering** into a single intelligent system.

AI-Powered TikZ Generation – Instead of templates or drag-and-drop tools, the system uses IBM Granite to reason over plain-English input and generate syntactically correct, publication-ready TikZ code autonomously.

Zero LaTeX Knowledge Required – Researchers with no TikZ experience can produce IEEE-quality diagrams in seconds, dramatically lowering the barrier to academic publishing.

Multi-Turn Refinement – Continuously adapts diagram output based on plain-English follow-up commands, maintaining full context across iterations without regenerating from scratch.

In-Notebook Live Preview – Renders the generated diagram as a visual image directly inside Watson Studio using matplotlib — no external tools, no Overleaf, no LaTeX compiler needed.

Multi-Diagram Type Support – Supports flowcharts, state machines, trees, block diagrams, timelines, Venn diagrams, and sequence diagrams from a single unified interface.

IBM Ecosystem Native – Built entirely within IBM Watson Studio and IBM watsonx.ai, ensuring reliability, AI governance, and enterprise-grade scalability with **IBM Granite models**.

This makes the solution a **smart, adaptive, and autonomous diagram assistant**, not just a static code generator — enabling researchers to focus on their work while the AI handles the LaTeX complexity.

Git Hub Link

·Github URL –

https://github.com/0merfarooq/IBM_SKILLSBUILD_LaTex_Generation_AI_Project_Problem_Statement_26

https://github.com/0merfarooq/IBM_SKILLSBUILD_LaTex_Generation_AI_Project_Problem_Statement_26

·

·

·

Future Scope

1.Overleaf & LaTeX IDE Integration –

The system can be extended with a direct Overleaf API integration, allowing generated TikZ code to be pushed straight into a user's Overleaf project with a single command. This will enable researchers to go from plain-English description to a compiled, rendered PDF document entirely without leaving their LaTeX workflow.

2.Sketch-to-Diagram (Image Input) –

Enhancing the system with image understanding capabilities will allow users to upload a hand-drawn sketch or a rough diagram photo. The Granite vision model can interpret the sketch and generate corresponding TikZ code automatically, making the tool accessible even to users who prefer drawing over typing.

3.Expanded Diagram Type Library –

Future versions can support a broader range of academic diagram types including UML class diagrams, entity-relationship (ER) diagrams, circuit schematics, neural network architectures, and mathematical function plots — covering all major use cases across computer science, engineering, and mathematics research.

4.Gradio / Streamlit Web UI Deployment –

The notebook-based interface can be upgraded to a fully deployed web application using Gradio or Streamlit, hosted on IBM Cloud. This will allow researchers to access the diagram generator from any browser without requiring a Watson Studio account or any local setup.

5.RAG-Based TikZ Knowledge Retrieval –

Integrating a Retrieval-Augmented Generation (RAG) pipeline with a vector database of TikZ documentation, examples, and academic diagram templates will significantly improve code accuracy and output quality. The Granite model will retrieve relevant TikZ patterns before generating, reducing syntax errors and improving diagram complexity handling.

6.Multi-Language LaTeX Support –

Extending the system to support PGFPlots, Asymptote, and MetaPost alongside TikZ will make it useful for a wider academic audience, including those working in mathematics, physics, and scientific data visualization who rely on different LaTeX diagram packages.

In recognition of the commitment to achieve
professional excellence



Mohammed Omer Farooq

Has successfully satisfied the requirements for:

Getting Started with Artificial Intelligence



Issued on: May 25, 2026
Issued by: IBM SkillsBuild

Verify: <https://www.credly.com/badges/65e3b1de-0dc9-4de0-a014-1bd6c88d3677>





This certificate is presented to
Mohammed Omer Farooq

for the completion of
Lab: Troubleshoot Your Code Using IBM Bob
(ALM-COURSE_4071307)

According to the Adobe Learning Manager system of record

Thank You!

Thank you for your time and interest.